
Programmation des Architectures Parallèles

Rapport étape 2 : projet-06

16 mars 2026

1. - Introduction et principes de l'évaluation paresseuse

1.1 - Objectifs de l'étape 2

Lors de la première étape, nos calculs s'appliquaient aveuglément à l'ensemble du domaine, qu'il y ait de l'activité ou non. Or, dans des configurations comme `alea` ou en fin de stabilisation pour les autres configurations, d'immenses zones de la grille sont parfaitement stables.

L'objectif de cette seconde étape est d'implémenter un mécanisme d'**évaluation paresseuse**. Dans ce cadre, le noyau ne doit recalculer que les tuiles contenant un potentiel d'instabilité. Pour cela, nous utilisons deux structures. La première est un **Quadtree** permettant de suivre et d'élaguer efficacement de larges régions spatiales inactives de la grille. La deuxième est un tableau d'entiers en tant que tableau de booléens (**Bitset**). Les spécificités de l'implémentation de ces deux structures sont disponibles en détails en annexe A. Nous allons tout de même expliquer plus bas une petite partie de leur fonctionnement qui est nécessaire pour comprendre les choix faits et les résultats obtenus dans ce rapport.

1.2 - Configuration matérielle et contraintes hétérogènes

Rappelons que les processeurs de la salle 104 possèdent une architecture hybride (8 P-Cores rapides avec cache L1 dédié, 16 E-Cores plus lents en clusters partageant leur cache L2 par groupe de 4). Cette dissymétrie peut avoir une influence sur nos décisions de placement de threads (`OMP_PLACES`) et sur nos choix d'ordonnancement (*scheduling* dynamique vs statique) lorsque la charge de travail paresseuse deviendra très irrégulière.

2 - Stratégie séquentielle paresseuse et optimisation interne

Les deux versions du problème ont été optimisées avec des méthodes différentes, que ce soit au niveau de la structure de donnée utilisée pour la gestion paresseuse, ou des optimisations de traitements des tuiles.

2.1 - Traitement asynchrone (`asandpile`)

2.1.1 - Le Quadtree local

Pour éviter les parcours inutiles, nous avons implémenté un Quadtree sous forme de tableau optimisant la localité mémoire des caches et simplifiant le calcul des indexes. Il utilise un mécanisme de double-buffering (`cur` et `next`) pour séparer les marquages de statut modifiés durant l'itération courante de ceux valables pour l'itération suivante (voir annexe A).

2.1.2 - Stabilisation interne et détection de bords

L'évaluation paresseuse naïve réveille souvent artificiellement les tuiles voisines. Pour le noyau asynchrone, nous avons développé le traitement `asandPile_do_tile_opt_lazy` avec une boucle de **stabilisation interne** (limitée par un `MAX_INNER_PASSES`) et 3 valeurs en retour :

- `TILE_STABLE` : rien n'a bougé.
- `TILE_INTERNAL_CHANGE` : des grains se sont écoulés, mais aucun n'a franchi la bordure. On marque uniquement la tuile courante dans le quadtree.
- `TILE_BORDER_CHANGE` : un grain a quitté la tuile, on doit marquer la tuile et ses 4 voisines directes dans le quadtree.

Listing 1 – Extrait : Logique de stabilisation et détection de passage de bordure

```

1 do { // boucle de stabilisation interne
2     local_change = 0;
3     for (int i = y; i < y + height; ++i) {
4         /* ... balayage de la tuile ... */
5         if (__builtin_expect (val >= 4, 0)) {
6             /* ... ebolement ... */
7             local_change = 1; any_change = 1;
8             // Seules les cellules sur le contour propagent aux tuiles voisines
9             if (j == 0 || j == width - 1 || i == y || i == y + height - 1)
10                 border_change = 1;
11         }
12     }
13     pass++;
14 } while (local_change && pass < MAX_INNER_PASSES);

```

2.2 - Traitement synchrone (ssandPile)

Nous allons traiter de l’implémentation synchrone utilisée pour le problème 1024x1024 avec une configuration en spirales.

2.2.1 - Bitset

L’évaluation paresseuse du problème en version synchrone utilise la structure de Bitset détaillée en annexe B. En résumé, le Bitset utilise un tableau d’entiers de 64 bits où chaque bit de chaque entier représente l’état d’une tuile. On regroupe ces entiers par ligne de tuile. Ainsi, en prenant comme exemple une taille de 1024, pour des tailles de tuile de 16x8, nous obtenons 128 lignes de 64 tuiles. Ce qui est représenté par un tableau de 128 entiers. Dans le cas où une ligne possède plus de 64 tuiles, plusieurs entiers peuvent être utilisés pour représenter l’entièreté de la ligne.

2.2.2 - Comptages et récupération des tuiles à traiter

L’utilisation d’entiers comme tableau de booléens permet une récupération extrêmement rapide des tuiles à traiter. En l’occurrence, nous maintenons un tableau PER_SET contenant le même nombre de cases qu’il y a d’entiers dans notre bitset. Ce tableau contient, pour chaque case *i*, le nombre de tuiles marquées dans l’entier *i* du bitset. Ainsi, la double boucle du parcours des tuiles devient une double boucle sur les entiers, puis sur le nombre de tuiles présentes dans cet entier. Enfin, La récupération de chaque tuile à traiter se fait à partir de l’opération de récupération du bit le plus à gauche (en $O(1)$). Cela empêche un parcours des 64 bits s’il n’y a que peu de tuiles dans le bitset. Enfin, on ne modifie plus la variable *changed* directement dans la boucle. Par simplicité, on préfère compter le nombre de cases à la fin, la variable *changed* correspond à une simple vérification sur le nombre de tuiles restantes à la fin d’une itération.

Listing 2 – Extrait : Boucle d’itérations (simplifiée)

```

1 for (unsigned it = 1; it <= nb_iter; it++) {
2     ...
3
4     for (unsigned i = 0; i < TOTAL_NB_SETS; i++) {
5         ...
6
7         for (unsigned char k = 0; k < in_set; k++) {
8             ...
9
10            int loc_change = do_tile (x + x_0, y + y_0, TILE_W - x_end - x_0,
11                                     TILE_H - y_end - y_0);
12
13            tile self = t;
14            tile up = {.tx = t.tx, .ty = t.ty - 1 + y_0};
15            tile down = {.tx = t.tx, .ty = t.ty + 1 - y_end};
16            tile left = {.tx = t.tx - 1 + x_0, .ty = t.ty};
17            tile right = {.tx = t.tx + 1 - x_end, .ty = t.ty};
18
19            tileset_mark_at (TILESET, loc_change, self);
20            tileset_mark_at (TILESET, loc_change, up);
21            tileset_mark_at (TILESET, loc_change, down);
22            tileset_mark_at (TILESET, loc_change, left);

```

```

23     tileset_mark_at (TILESET, loc_change, right);
24 }
25 }
26
27 changed = tileset_count(TILESET);
28 if(!changed)
29     return it;
30 }

```

2.2.3 - Traitement des bords

Le traitement des bords sur la version synchrone se fait complètement différemment de la version asynchrone. Nous définissons, dans l'entier renvoyé par `do_tile`, plusieurs flags :

Flag	Signification	Valeur
BORDER_TILE	Un changement s'est produit dans la tuile	1
BORDER_UP	Un changement s'est produit sur la bordure haute	1 « 1
BORDER_DOWN	Un changement s'est produit sur la bordure basse	1 « 2
BORDER_LEFT	Un changement s'est produit sur la bordure gauche	1 « 3
BORDER_RIGHT	Un changement s'est produit sur la bordure droite	1 « 4

FIGURE 1 – Tableau des flags utilisés pour la détection de bordures (ssandpile)

Par exemple, la bordure gauche ne sera alors marquée que lorsque `do_tile` renverra un entier dont le flag gauche est à 1. Le flag de changement interne à la tuile a pour valeur 1 afin de pouvoir assurer la compatibilité avec les versions de compute ne gérant pas les bords (et qui attendent souvent simplement que le retour soit différent de 1, ou au moins différent de 0). Cela nous permet de vérifier l'impact du placement des flags sur les performances, dont nous discuterons juste après.

2.3 - Résultats et comparaisons

Le plus gros facteur sur les performances séquentielles, pour l'évaluation lazy, est la taille des tuiles. Nous avons donc effectué des tests via des heatmaps.

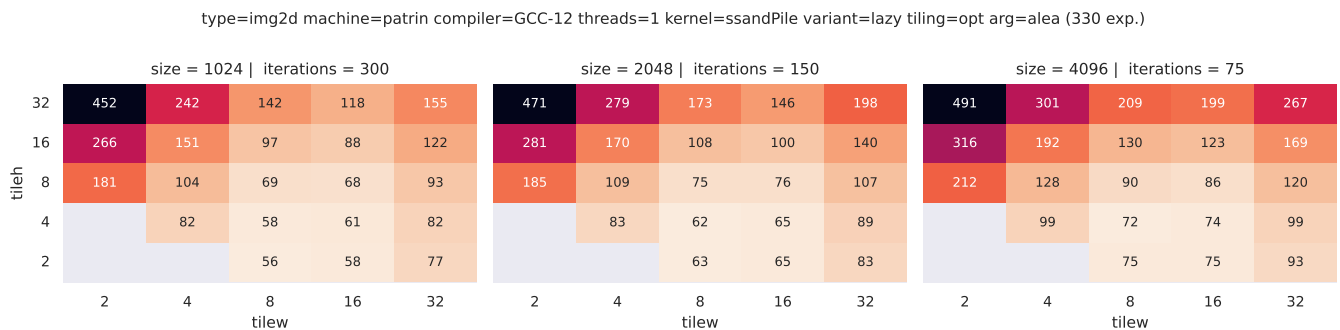


FIGURE 2 – Heatmap de tailles de tuiles pour 1024, 2048 et 4096

Il est important de noter que, bien que ces graphiques utilisent le bitset, les résultats sont similaires voir identiques pour les tailles des tuiles du Quadtree. Ainsi, nous nous baserons sur ces tailles pour les deux structures. On peut donc voir que les meilleures tailles de tuiles se retrouvent entre 16x2 et 16x8. Dans nos tests, nous avons aussi remarqué que 16x16 est un bon candidat pour certaines configurations.

Nous voulons maintenant confirmer les gains séquentiels de nos optimisations lazy. Nous testons ici des tailles différentes pour les 2 implémentations, car nous ne les comparons pas entre elle. Sur la version synchrone, nous testons aussi le marquage selon les bords, via le noyau `lazy_border`. On remarque alors la différence de performances. Il est bon de noter que ces expériences sont importantes pour comprendre si nos futurs gains sont dû à nos structures ou bel et bien à un bon parallélisme.



FIGURE 3 – Gain séquentiel : asandpile/Quadtree (gauche), ssandpile/Bitset (droite)

3 - Parallélisation asynchrone paresseuse (asandPile)

3.1 - Stratégies de coloriage et prévention des conditions de course

Le noyau asynchrone modifiant la grille en place, traiter des tuiles adjacentes simultanément provoquerait des conditions de course sur les bordures partagées. Pour y remédier, nous utilisons des approches de coloriage :

1) approche générale (4 couleurs) : pour des configurations de grande taille (2048 spirals, 8192 alea), la grille est découpée en tuiles associées à 4 couleurs distinctes. Ainsi, deux tuiles de même couleur sont séparées par au moins une tuile, empêchant deux threads de modifier des cellules partagées, incluant les diagonales sur les coins. Chaque itération nécessite donc 4 boucles successives, séparées par une barrière de synchronisation OpenMP implicite;

2) approche spécifique au cas dense (2 couleurs pour le 256 4partout) : sur une très petite grille de 256×256 avec une saturation d'activité, le coût des 4 barrières de synchronisation par itération devient un goulot d'étranglement. Nous avons implémenté une fonction `omp_2coloring` qui découpe la grille en bandes horizontales (assignée par thread). La couleur 0 correspond aux bandes paires, la couleur 1 aux bandes impaires. Cette stratégie ne présente pas de condition de course parce que le modèle du tas de sable est 4-connecté, un grain instable s'écoulant au maximum d'une case vers le nord, le sud, l'est ou l'ouest à chaque pas de simulation interne. Il n'atteindra donc jamais la bande traitée par un autre thread. Diviser par deux le nombre de phases de coloriage a doublé le parallélisme disponible par phase et divisé par deux l'overhead récurrent des barrières, ce qui s'est avéré positif sur ce cas précis. Compte-tenu de ces observations, il est aisé de remarquer l'intérêt d'un ordonnancement `static, 1` pour partager les bandes entre les threads, tout en privilégiant l'utilisation des E-cores pour homogénéiser la vitesse de traitement d'une bande (divisée en petites tuiles au temps de traitement négligeable) tel qu'on le verra plus tard.

Pour les cas clairsemés ou asymétriques (8192 alea, 2048 spirals) nécessitant le Quadtree, l'écriture concourante de l'état `DIRTY` d'une tuile voisine est gérée sans protection ni verrous. Elle repose sur l'idempotence de l'affectation en C (les threads concurrents se contentant de sur-écrire la valeur 1 au même octet mémoire). Ces collisions bénignes permettent de s'affranchir d'instructions atomiques coûteuses.

3.2 - Expérimentations selon les typologies de configurations

Nous avons isolé trois cas d'école différents (fournis à des fins de tests de performance) nécessitant des stratégies d'ordonnancement, de placement et d'algorithmiques adaptées. Pour appuyer nos choix, et dans la perspective de donner une vue d'ensemble sur le problème asynchrone, nous avons étudié la politique d'ordonnancement relative au cas 256, l'impact des politiques de placement dans le cas 2048, et le choix de la taille de tuile dans le cas 8192. Cette spécialisation des expériences pour chaque taille d'automate nous permet de balayer un large spectre de problématiques relatives à leurs spécificités, l'exposition du cheminement scientifique pour chaque instance étant hors de portée du présent rapport.

a) Cas dense : 256×256 , configuration 4partout

Le modèle est inondé d'activité de manière homogène. Le Quadtree reste presque totalement invalide jusqu'à la fin de la simulation. L'évaluation paresseuse pure génère donc plus d'overhead que de gains. Nous avons opté pour le noyau spécifique `omp_2coloring` discuté en section 3.1, ce dernier n'utilisant pas d'évaluation lazy.

type=img2d machine=shaddam compiler=GCC-12 size=256 kernel=asandPile tiling=opt iterations=1000 arg=4partout refTime=134
tile_size=16×16 (492 exp.)

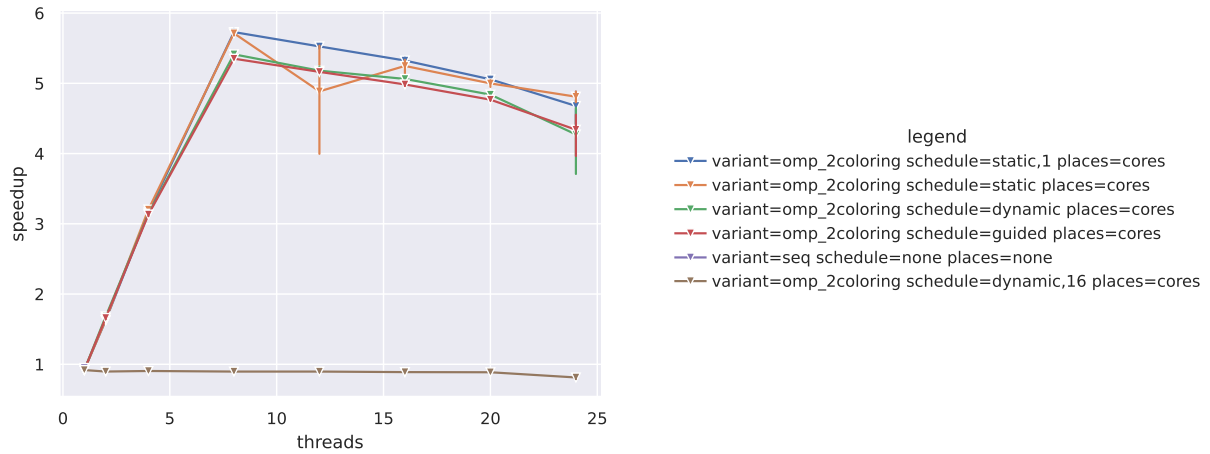


FIGURE 4 – Évolution du speedup pour le noyau `omp_2coloring` (256×256 , tuiles 16×16) selon différentes politiques d'ordonnancement.

L'analyse de la figure 4 valide nos hypothèses matérielles et algorithmiques. Toutes les courbes d'ordonnement fin (*static*, *1*, *dynamic*, etc.) culminent à 8 threads (speedup $\approx 5,8\times$) avant de décliner légèrement. Dans ce noyau à 2 couleurs, nous itérons par groupes de lignes horizontales, et avec une grille de largeur 256 et des tuiles de hauteur 16, chaque phase de couleur contient exactement $\frac{256}{2 \times 16} = 8$ bandes indépendantes. Le parallélisme maximal est donc de 8 tâches, qu'il est optimal de confier aux 8 P-Cores rapides. Au-delà, les E-Cores se réveillent inutilement sans trouver de travail, générant un surcoût de synchronisation. Ce plafond est confirmé a contrario par la courbe *dynamic*, *16*, qui stagne à un speedup de $\approx 1\times$ car en groupant les tâches par blocs de 16, un seul thread accapare d'emblée les 8 bandes disponibles et l'exécution se retrouve intrinsèquement sérialisée. Forts de ce constat de granularité, nous appliquons pour les grandes topologies une politique de distribution fine et dynamique afin d'éviter la sous-utilisation des cœurs ou l'overhead de verrous : *static*, *16* pour le cas 2048 (taille intermédiaire où les lignes présentent des charges asymétriques) et *dynamic*, *256* pour le cas 8192.

D'ailleurs, augmenter le parallélisme (générer plus de bandes) afin d'obtenir un problème parfaitement partagé sur plus de 8 threads peut rendre les P-cores inutiles. Ces derniers devront attendre la fin de l'exécution des E-cores pour passer à leur prochaine itération. Modulo les soucis de cache, une configuration avec 8 P-cores et 8 E-cores, devient alors équivalente à une configuration utilisant 16 E-cores.

b) Cas asymétrique : 2048×2048 , configuration *spirals*

L'ordonnement *static*, *16* et les tuiles 16×16 ayant été sélectionnées au préalable, nous avons réalisé une analyse de placement pour déterminer si la topologie big/little influence les performances.

type=img2d machine=moneo compiler=GCC-12 size=2048 kernel=asandPile iterations=300 arg=spirals refTime=39 tile_size=32×32 (765 exp.)



FIGURE 5 – Speedup du noyau `omp_lazy_bidi` (2048×2048 , spirals, tuiles 32×32) selon la politique de placement `OMP_PLACES`.

La figure 5 met en évidence plusieurs comportements. La configuration `places=8:16` (E-Cores seuls) plafonne à un speedup de $\approx 3\times$ pour 12–14 threads, inférieur aux autres variantes : les E-Cores partagent leur cache L2 par clusters de 4 (4 Mo pour 4 cœurs), ce qui génère de la contention lors des accès aux bordures de tuiles voisines traitées par des threads du même cluster. Les variantes reposant sur les P-Cores (`cores`, `threads`) ou mixtes atteignent quant à elles un speedup similaire de $\approx 4\times$ autour de 20–22 threads, illustrant que les P-Cores, chacun doté d’un L2 de 3 Mo dédié, excellent sur des tuiles indépendantes sans contention de cache. La convergence des courbes `cores` et `threads` au-delà de 8 threads montre que l’ajout d’E-Cores ne dégrade pas les performances si le work-stealing du scheduler équilibre la charge entre les cœurs hétérogènes. En complément de l’ordonnancement, nous avons implémenté un traitement *boustrophédon* (ou balayage bidirectionnel) pour pallier le défaut de propagation asymétrique. Dans la configuration `spirals`, l’activité se propage en spirale, rendant le balayage standard (haut-gauche vers bas-droite) très lent pour faire remonter l’information vers le haut de la grille. Le balayage bidirectionnel, détaillé en annexe C, alterne les directions de parcours au sein de chaque tuile pour accélérer la stabilisation locale et réduire le nombre d’itérations globales nécessaires.

c) cas clairsemé : 8192×8192 , configuration `alea`

La configuration `alea` sur une grille de 8192×8192 est le cas idéal pour l’évaluation paresseuse. Seule une fraction de la grille est active à chaque instant, le Quadtree élaguant alors convenablement les zones stables.

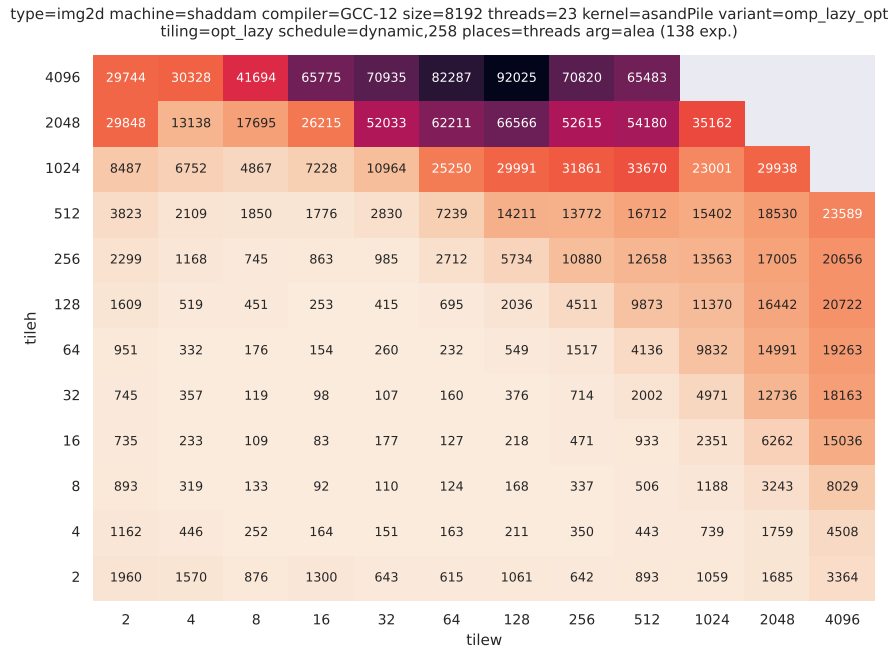


FIGURE 6 – Heatmap des temps d’exécution (ms) pour `omp_lazy_opt` sur le cas 8192×8192 `alea`, avec 23 threads, `OMP_SCHEDULE=dynamic, 258`.

L’observation de la figure 6 révèle plusieurs régimes que l’on peut expliquer par des mécanismes matériels. Les meilleures performances apparaissent pour des tuiles de 16×16 ou 16×8 (83–92 ms). Une tuile 16×16 d’`unsigned int` occupe $16 \times 16 \times 4 = 1$ Ko, ce qui tient largement dans le cache L1 (~ 32 Ko par P-Core). Le contenu de la tuile et ses bordures immédiates nécessaires aux opérations d’éboulement restent donc en L1, limitant fortement les défauts de cache. De plus, la grille 8192×8192 est alors découpée en $512 \times 512 = 262\,144$ tuiles, permettant au Quadtree d’effectuer un élagage précis, un grain actif n’activant que quelques tuiles voisines.

Lorsque les tuiles deviennent très petites (2×2 , 4×4), les performances se dégradent malgré leur excellent comportement cache. La grille contient alors jusqu’à ~ 16 millions de tuiles, et le coût de gestion du Quadtree (parcours, marquage, empilement) dépasse largement celui du calcul des cellules. L’ordonnancement `dynamic, 258` devient également moins efficace, car le scheduler OpenMP doit gérer un très grand nombre de tâches.

À l’inverse, les grandes tuiles (256×256 et plus) entraînent aussi une dégradation. Une tuile 256×256 représente 65 536 cellules (256 Ko), dépassant la capacité du cache L1 et occupant une part non négligeable du cache L2. Les accès aux bordures provoquent donc davantage d’évictions. Surtout, dans un cas `alea`, un unique grain actif peut marquer toute la tuile comme « dirty », forçant le recalcul de dizaines de milliers de cellules stables et réduisant fortement la précision du Quadtree.

4 - Parallélisation synchrone paresseuse (ssandPile)

Pour rappel, nous utilisons ici un Bitset. La parallélisation du problème est, ironiquement, rendue beaucoup plus difficile par le fait que nous rassemblons les tuiles dans une seule et même donnée, un entier. Pour notre version parallèle, nous avons décidé de ne distribuer les threads que sur les entiers en questions. Plus simplement, les threads travaillent sur des paquets de, au maximum, 64 tuiles. Pourtant ces paquets ne possèdent pas forcément de travail réel (si le set est à 0). Cela provoque parfois des situations où certains threads ne travaillent absolument pas, ce qui se verra au long de cette partie. Nous discuterons en dernier des possibles améliorations liée à cette distribution inégale.

4.1 - Première approche : blocage lors du marquage

L'implémentation parallèle faite à l'origine se basait sur le blocage, au moyen d'un `omp critical`, lors du marquage d'une nouvelle tuile :

Listing 3 – Extrait : marquage d'une tuile dans la version parallèle originale (dans la boucle d'itérations)

```
1  #pragma omp parallel
2  for (unsigned i = 0; i < TOTAL_NB_SETS; i++) {
3      ...
4
5      for (unsigned char k = 0; k < in_set; k++) {
6          ...
7
8          int loc_change = do_tile (x + x_0, y + y_0, TILE_W - x_end - x_0,
9                                   TILE_H - y_end - y_0);
10
11         tile self  = t;
12         tile up    = {.tx = t.tx, .ty = t.ty - 1 + y_0};
13         tile down  = {.tx = t.tx, .ty = t.ty + 1 - y_end};
14         tile left  = {.tx = t.tx - 1 + x_0, .ty = t.ty};
15         tile right = {.tx = t.tx + 1 - x_end, .ty = t.ty};
16
17     #pragma omp critical
18     {
19         tileset_mark_at (TILESET, loc_change, self);
20         tileset_mark_at (TILESET, loc_change, up);
21         tileset_mark_at (TILESET, loc_change, down);
22         tileset_mark_at (TILESET, loc_change, left);
23         tileset_mark_at (TILESET, loc_change, right);
24     }
25 }
26 }
```

A noter que nous n'avons pas besoin de modifier la variable `changed`, car nous utilisons dans tous les cas fonction de comptage comme décrit en annexe B. Cette approche s'est révélée catastrophique, et rendait les performances pires que la version séquentielle. Pour comprendre le problème, nous avons donc observé la trace produite par l'exécution du code (à noter que pour cause de stabilité à distance, les traces sont faites avec les E-cores uniquement).

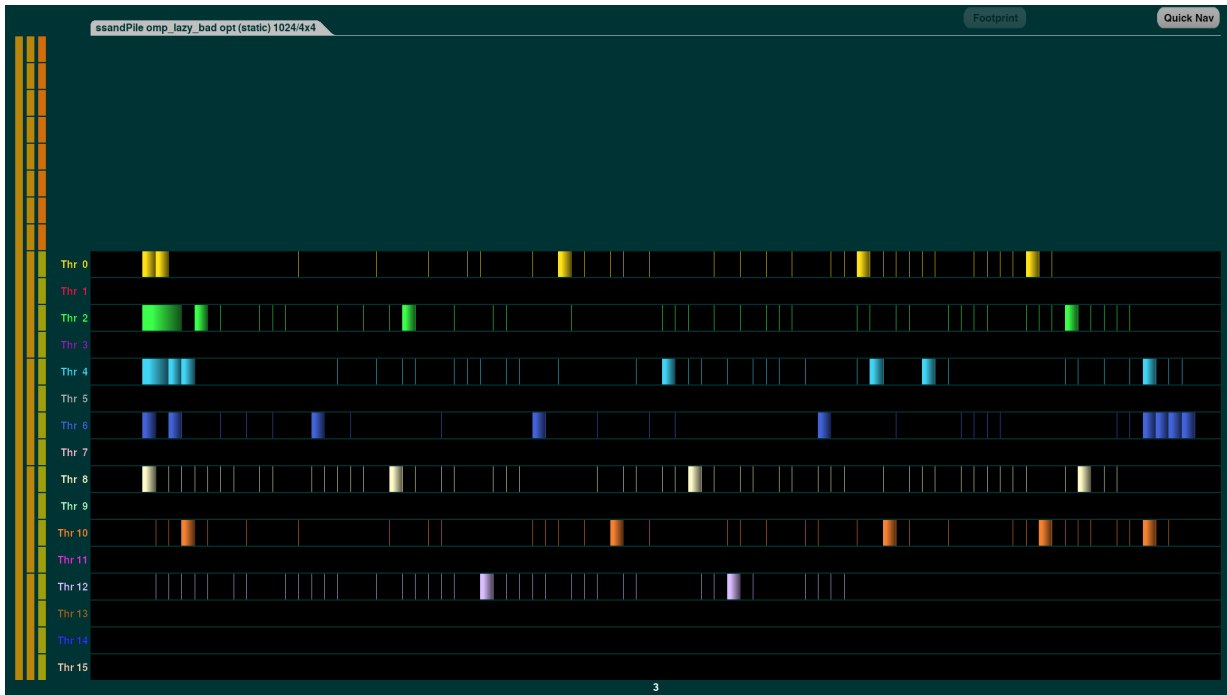


FIGURE 7 – Trace de la première version multithreadée du problème en 1024x1024

On peut en déduire que nos tailles de tuiles (exagérées dans cet exemple, bien que proches de notre usage) font que les threads se retrouvent souvent à exécuter la partie de marquage du code. Sachant que nous marquons les voisins de la tuile courante, nous ne pouvons pas non plus garantir qu'un autre thread ne touche pas au passage à une tuile que nous marquons. En l'occurrence, les tuiles sont marquées au moyen d'un OU (|) binaire, et les cases du Bitset sont touchées que le marquage les rende vraies ou non. Ainsi, nous ne pouvons pas non plus laisser les cases non protégées, au risque de perdre certaines tuiles.

4.2 - Approche "Bitset local"

Considérant que notre Bitset ne prend en réalité que peu d'espace en mémoire (c.f empreinte mémoire en annexe B), il n'est pas très problématique de recréer des bitsets locaux à chaque threads. Ainsi, chaque thread marquerait dans son propre Bitset les tuiles non stables, et il suffirait d'une passe de rassemblement à la fin.

Ainsi, on peut complètement supprimer le critical de notre boucle. Le code change alors légèrement :

Listing 4 – Extrait : marquage d'une tuile en parallèle avec des sets locaux à chaque thread (dans la boucle d'itérations)

```

1  #pragma omp parallel {
2      Tileset curr = tilesets[omp_get_thread_num()];
3  for (unsigned i = 0; i < TOTAL_NB_SETS; i++) {
4      ...
5
6      for (unsigned char k = 0; k < in_set; k++) {
7          ...
8
9          int loc_change = do_tile (x + x_0, y + y_0, TILE_W - x_end - x_0,
10                                   TILE_H - y_end - y_0);
11
12         // definition des tuiles self, up, down, left, et right
13         ...
14
15         tileset_mark_at (curr, loc_change, self);
16         tileset_mark_at (curr, loc_change, up);
17         tileset_mark_at (curr, loc_change, down);
18         tileset_mark_at (curr, loc_change, left);
19         tileset_mark_at (curr, loc_change, right);
20     }
21 }
22

```

```

23 }
24
25 changed = tileset_merge(TILESET, tilesets, num_threads);
26 if(!changed)
27     return it;

```

En revérifiant notre trace, on observe que les threads travaillent beaucoup plus :

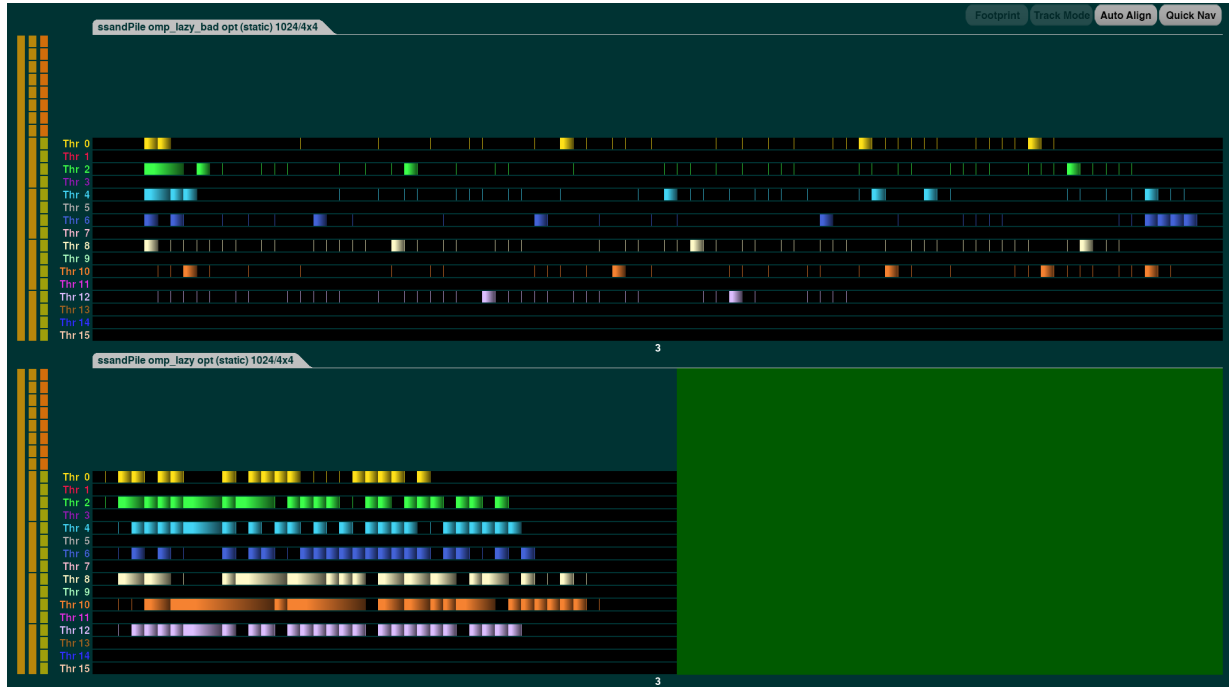


FIGURE 8 – Trace de comparaison avec synchronisation (en haut) contre sans synchronisation (en bas), 1024x1024

Ainsi, nous n'avons que l'overhead du merge en plus. Il est important de noter que désormais, il n'y a plus aucune synchronisation à l'intérieur de nos boucles de traitement de tuiles. Par conséquent, on peut supposer que les attentes des threads sont sûrement dues à des problèmes de caches ou de mauvais partage du travail. Nous avons au passage essayé de créer une version parallèle du merge, malheureusement nous n'avons pas observé de gains, étant donné que nous ne nous sommes pas concentrés sur ce problème, nous n'en parlerons pas plus en détails.

4.3 - Optimisation des paramètres

Nous avons directement comparé les courbes de speedup en l'état avant de s'attaquer aux autres paramètres. Il s'agit par ailleurs du plus intéressant des paramètres que nous ayons eu à traiter pour ce cas. Pour déterminer la taille de tuile à utiliser pour ces tests, nous nous sommes basé sur les résultats séquentiels et sommes partis sur du 16x4 (après comparaison pour cette configuration avec les autres). En suivant le cas de assandpile, nous avons tenté de comparer le speedup d'une version utilisant uniquement les E-cores d'une version utilisant les P-cores et les E-cores.

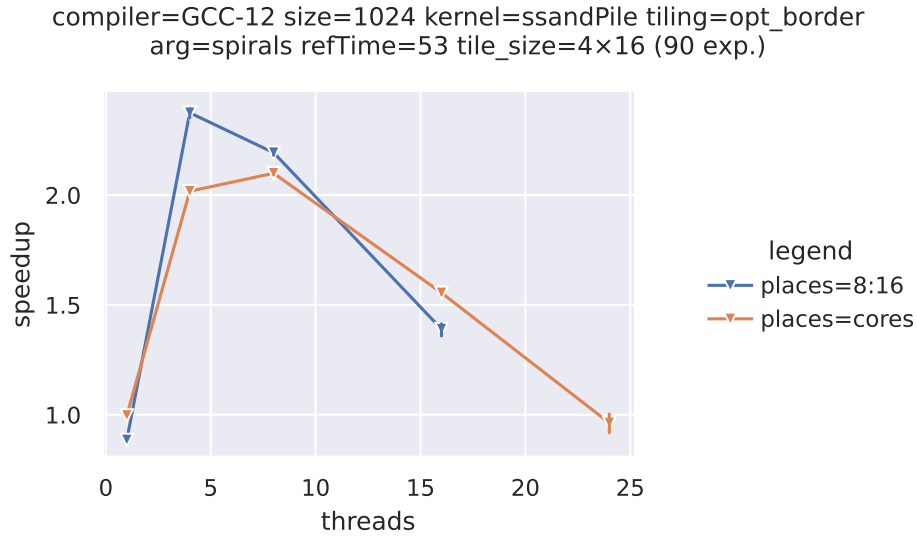


FIGURE 9 – Courbes de speedup 1024 ssandpile : E-cores et tous les coeurs.

Pour nos machines, mettre `OMP_PLACES` à 8 : 16 revient à demander la sélection des E-cores. Nous nous sommes rendus compte que l'utilisation de 4 E-cores groupés ensemble était plus efficace que l'utilisation de 4 P-cores. Notons que les tuiles utilisées sont extrêmement petites (16×4) et donc ne demandent que peu de traitement. On peut alors se douter que le temps d'exécution sera beaucoup plus dépendant de la vitesse de la mémoire que de la vitesse du processeur. Considérant cela, les P-cores perdent alors leur seul intérêt. Si on considère la taille du problème, soit $1024 \times 1024 \times 4 \times 2 = 8388608$ octets, et en considérant la nature de l'évaluation paresseuse qui fait descendre cette taille énormément (on peut supposer au moins par 2 au vue de la configuration), on peut donc supposer que la grande majorité des tuiles rentrent dans le cache L2 des E-cores (4Mo). Ce cache étant partagé entre les 4 coeurs, l'impact du changement du coeur traitant une tuile d'une itération à la suivante s'en retrouve grandement réduit. On peut s'assurer de cette idée au moyen d'une exécution de 4 E-cores dans des groupes séparés. En s'intéressant au cas spécifique à 4 coeurs, on peut alors comparer l'exécution selon la répartition des coeurs.

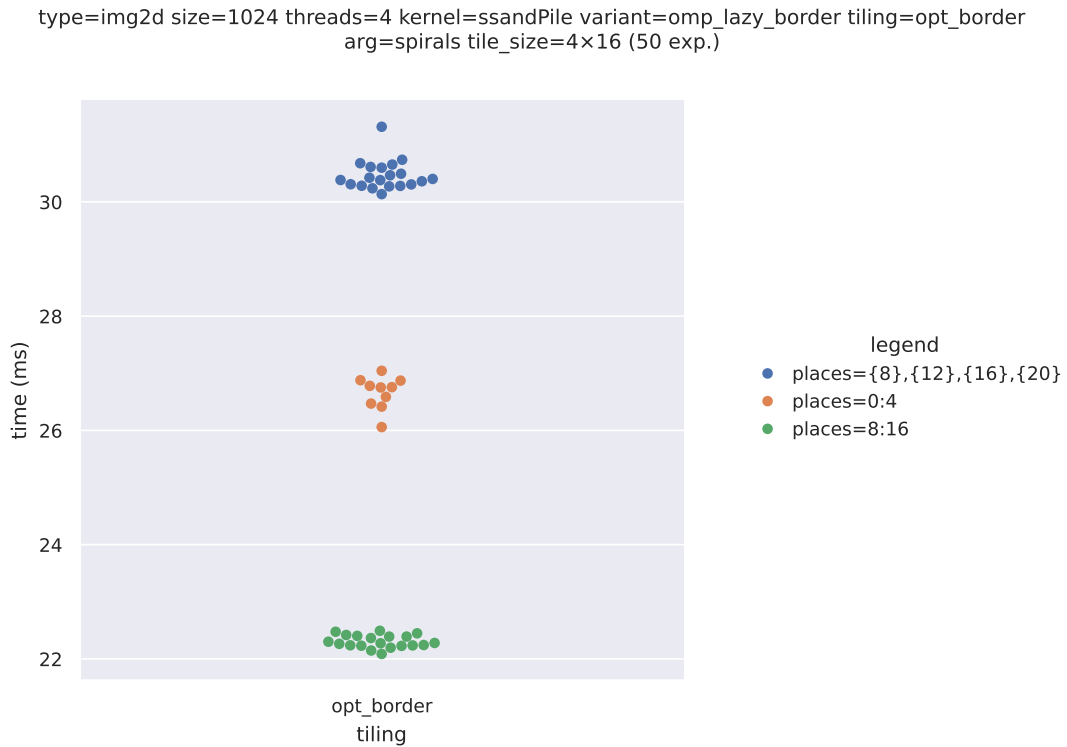


FIGURE 10 – Comparaison de l'exécution sur 4 coeurs selon la répartition des coeurs.

Ici, on choisit pour être sûr de placer à la main les coeurs. On remarque que les E-cores dispersés ont alors perdu tous les avantages qu’ils avaient par rapport aux P-cores, et performant alors beaucoup moins bien. Les E-cores regroupés restent eux plus rapides.

Enfin, il nous faut souligner que ce cas ne vaut que pour la configuration en spirale, qui est une configuration où le nombre de cases actives est assez bas. Nous avons pu faire quelques tests avec d’autres types de configurations. Dans ces cas, il pourra être valable d’utiliser plus de 4 coeurs. Nous en parlons comme ouverture, car nous n’avons pas poussé nos expérimentations sur les autres types de configurations via la résolution synchrone.

4.4 - Futures optimisations possibles

Comme dit précédemment, le partage de travail actuel souffre du fait que nous ne partageons que des paquets qui, potentiellement, ont de grosses différences dans leurs charges de travail. Nous avons réfléchi à une façon de corriger ces problèmes, qui pourra être ajoutée plus tard. L’idée serait, lors du merge, de créer une liste des tuiles à traiter au passage. Ainsi, nous n’aurions plus de double boucle mais une seule, itérant sur l’ensemble des tuiles à réellement traiter. Ce nouveau traitement aurait en plus une autre utilité, celle de rajouter une charge de travail dans la fonction de merge. Même si cela paraît contre productif, une plus grosse charge de travail signifierait qu’il serait alors peut-être valable de paralléliser cette fonction qui, jusque là, n’a pas bénéficié du partage de travail.

4.5 - Analyse de l’impact du compilateur

Afin de s’assurer que nos résultats et nos gains de performance ne sont pas fortement biaisés par les optimisations internes de **GCC 12** (compilateur utilisé par défaut lors de nos mesures), nous avons mené une évaluation comparative avec **Clang 14**. La figure 11 trace les temps d’exécution des deux compilateurs sur les trois cas isolés précédemment.

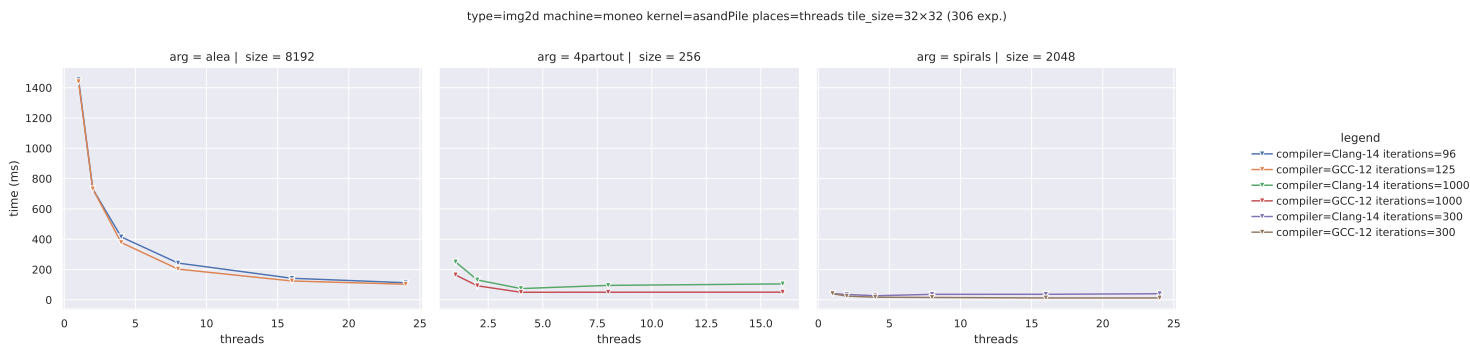


FIGURE 11 – Comparaison des temps d’exécution en ms selon le nombre de threads entre GCC-12 et Clang-14 sur les 3 configurations habituelles.

Les différences entre les deux compilateurs s’avèrent négligeables dans l’ensemble. Clang présente un très léger avantage en termes de temps d’exécution brut, visible particulièrement sur le cas dense (256 4partout). Cela suggère que ses stratégies de déroulage de boucle (*loop unrolling*) et de vectorisation s’adaptent un peu mieux à notre implémentation de tuile.

5 - Conclusion et perspectives

Tout d’abord, il faut préciser que nous n’avons pas pu discuter de tous les points sur lesquels nous avons travaillé durant cette étape. Ainsi, nous avons préféré nous concentrer sur les points que nous pensions être les plus intéressants. Parmi les points non abordés, nous ne sommes pas rentrés dans les détails de placement de nos données dans la mémoire, surtout avec les structures que nous avons créées, qui pouvaient donner des résultats différents selon le padding mémoire utilisé. Cette partie là est partiellement abordée dans la description des structures en annexe A, mais pas en détails. Nous n’avons pas parlé non plus par exemple des types de scheduling et de l’effet qu’a le fait de les donner statiquement dans le code contre le scheduling runtime. Nous avons par ailleurs encore plusieurs pistes d’optimisations, comme discuté précédemment dans ce rapport. Enfin, nous nous sommes donc concentrés sur la façon dont interagit notre code avec le cache et l’optimisation de l’utilisation de ce dernier, en l’occurrence, nous nous sommes rendus compte que l’utilisation de P-core s’avérait par moment inutile face à des coeurs plus lents mais dont les données sont partagées.

Annexes

A – Structure et implémentation du Quadtree paresseux

Le Quadtree est stocké sous la forme d'un tableau aplati d'octets `unsigned char`, représentant les nœuds de tous les niveaux de l'arbre. Chaque entrée vaut `QT_DIRTY` (1) ou `QT_CLEAN` (0).

Listing 5 – Structure LazyQT – `sandPile_common.h`

```
1 typedef struct {
2     int n; // dimension padded a la puissance de 2 sup.
3     int ntw, nth; // dimensions effectives de la grille de tuiles
4     int depth; // nombre de niveaux (feuilles = niveau depth-1)
5     int total_nodes; // total de noeuds sur tous les niveaux
6     unsigned char *cur; // flags "dirty" de l'iteration courante
7     unsigned char *nxt; // flags "dirty" prepares pour la prochaine iteration
8 } LazyQT;
```

L'indexation d'un nœud à un niveau ℓ , à la position (ty, tx) dans la grille de ce niveau se fait via :

Listing 6 – Calcul des index – inline helpers

```
1 static inline int qt_level_offset (int level) {
2     return ((1 << (2 * level)) - 1) / 3; // somme geom. 1+4+16+...
3 }
4 static inline int qt_node_index (int level, int ty, int tx) {
5     return qt_level_offset(level) + ty * (1 << level) + tx;
6 }
```

Pendant le calcul d'une itération, les threads lisent `cur` pour savoir si une tuile est à traiter, et écrivent dans `nxt` les marquages résultant des changements détectés. Après la boucle de calcul, `qt_swap_and_clear()` échange les deux tableaux (en swappant les pointeurs) et remet `nxt` à `QT_CLEAN` pour préparer l'itération suivante.

Dans la version séquentielle, l'arbre est parcouru récursivement de la racine vers les feuilles : un nœud interne `QT_CLEAN` court-circuite toute sa sous-arborescence, ce qui réduit le nombre de tuiles visitées à $\mathcal{O}(\text{tuiles actives} \cdot \log N)$.

Dans les versions parallèles (`omp_lazy_opt`, `omp_lazy_bidi`), la traversée récursive est abandonnée. Le scheduler OpenMP distribue les threads sur une double boucle `collapse(2)` plate couvrant toutes les tuiles candidates d'une couleur donnée. Chaque thread effectue un test `qt_is_leaf_dirty(tx, ty)` en $\mathcal{O}(1)$ (simple accès tableau sur `cur`) et saute la tuile si elle est propre. Cette approche supprime la récursion et permet au scheduler de répartir dynamiquement les tuiles actives entre les threads sans coordination préalable. En contrepartie, les tuiles propres sont quand même énumérées (le `continue` est atteint), ce qui est acceptable puisque le test est un simple accès mémoire.

Lorsqu'un changement de bordure est détecté après le calcul d'une tuile, la tuile et ses quatre voisines sont marquées `QT_DIRTY` dans `nxt` via `qt_mark_dirty_with_neighbors`. Un simple changement interne marque uniquement la tuile elle-même via `qt_mark_one_dirty_nxt`.

B – Structure et implémentation du Bitset paresseux

Le Bitset est un tableau d'entiers, où chaque entier représente au maximum 64 tuiles d'une ligne. Pour traverser l'ensemble des tuiles d'un bitset, on peut compter au moyen d'un `popcount` le nombre de tuiles dans un entier, et les traverser en récupérant itérativement les bit de poids fort et en les enlevant du set. Pour encoder l'entièreté d'une ligne, il se peut que plusieurs entiers soient utilisés. Lors de l'initialisation du bitset, on calcule le nombre de set nécessaire par lignes, le nombre de lignes, et le nombre de tuiles.

On utilise le terme **Bitset** dans ce rapport pour désigner l'ensemble des entiers encodant la grille, dans le code, un **bitset** est un entier, et l'ensemble des entiers s'appelle un **tileset**.

Listing 7 – Structure tileset – `sandPile_common.h`

```
1 struct _tileset
2 {
3     bitset *restrict sets;
4 };
5
6 typedef struct _tileset *restrict tileset_t;
```

A noter que, si la structure ne contient qu'un seul élément, il s'agit simplement d'un moyen pour nous de tester différentes façons d'encoder le Bitset. A la base, cette structure contenait plusieurs champs qui ont à terme été enlevés.

Listing 8 – Structure `tileset` : paramètre globaux – `sandPile_common.h`

```
1 extern tileset_t TILESET;
2 extern char *restrict PER_SET;
```

La variable `TILESET` définit le set global de la même façon que `TABLE` pour `sandpile`. La variable `PER_SET` définit le nombre de tuiles dans un seul et même entier. Il s'agit d'un buffer permettant d'éviter de compter réellement le nombre de tuiles dans chaque entier de la variable de bitset globale. Ces variables doivent être initialisées pour pouvoir utiliser le bitset.

Listing 9 – Structure `tileset` : paramètre globaux – `sandPile_common.h`

```
1 extern bitset TRUNC_MASK;
2 extern unsigned SETS_PER_ROW, TILES_PER_ROW, NB_ROWS, TOTAL_NB_SETS;
```

A l'initialisation de chaque bitset, ces paramètres sont au passage affectés. Dans le cas où l'on crée plusieurs bitset, on réaffecte les variables déjà affectées. Les bitsets sont pensés pour être utilisé pour des mêmes tailles, on ne gère donc pas le cas où l'on aurait des tailles différentes.

Pour utiliser les bitsets avec un code parallèle, il est préférable de créer un bitset par thread et d'utiliser les fonctions associées à la structure pour rassembler le résultat.

Plusieurs fonctions sont utilisables avec le bitset, les principales sont :

Listing 10 – Structure `tileset` : fonction de marquage – `sandPile_common.c`

```
1 void tileset_mark_at (tileset_t tileset, int change, const tile t)
2 {
3     unsigned long long tx = t.tx;
4     unsigned long long ty = t.ty;
5     unsigned long index = ty * SETS_PER_ROW + (BITSET_SIZE_DIV (tx));
6     bitset bit = bitset_at (BITSET_SIZE_MOD (tx));
7
8     tileset_at (tileset, index) |= bit * change;
9 }
```

La fonction de marquage d'un bit, soit une tuile, dans le set. On suppose que la tuile est une tuile correcte pour le set.

Listing 11 – Structure `tileset` : fonction de comptage – `sandPile_common.c`

```
1
2 inline int tileset_count (tileset_t tileset)
3 {
4     int left = 0;
5
6     for (unsigned i = 0; i < TOTAL_NB_SETS; i++) {
7         PER_SET [i] = __builtin_popcountll (tileset_at (tileset, i));
8         left |= (PER_SET [i] != 0);
9     }
10
11     return left;
12 }
```

Une fonction de comptage des tuiles marquées.

Listing 12 – Structure `tileset` : fonction de merge – `sandPile_common.c`

```
1 int tileset_merge (tileset_t tile, tileset_t *restrict others,
2                 unsigned nb_others)
3 {
4
5     for (unsigned i = 0; i < nb_others; i++)
6         for (unsigned j = 0; j < TOTAL_NB_SETS; j++)
7             tileset_at (tile, j) |= tileset_at (others [i], j);
8
9     return tileset_count (tile);
10 }
```

Une fonction de merge permettant le rassemblement de différents bitsets en un.

Un bitset étant un tableau d'entiers, leur poids en mémoire est relativement faible. On Par exemple, pour le problème ssandpile tel que l'on l'avons actuellement implémenté, soit en 1024x1024, avec des tuiles de tailles 16x4, la taille d'un bitset peut être représentée comme :

$SETS_PER_TILES = \frac{1024}{64 \times 16} = 1$ (nombre d'entiers pour encoder une ligne).

$NB_LIGNES = \frac{1024}{4} = 256$ (nombre de lignes).

$SIZE = 4 \times 1 \times 256 = 1024$ octets.

Dans notre cas, cela revient à $1024 \times 5 = 5120$ octets pour l'entièreté des bitsets alloués.

Enfin, nous avons passé beaucoup de temps à observé l'effet de la disposition de la mémoire des bitsets sur les performances. Ces tests reposaient sur deux valeurs à la compilation :

Listing 13 – Structure tileset : Configurations mémoire – sandPile_common.c

```
1 #define TILESET_MEM_PADDING 64 /* Padding between the allocated memory */
2 #define TILESET_MEM_ALIGN 1 /* Try to align the values of the struct */
```

Le premier ajoutant du padding entre certaines valeur de la structure. Le second permettant de forcer l'alignement de la mémoire selon le type de chaque variables dans la structure. La structure ayant désormais été réduite à une simple variable, leurs effets sont limités, non tenons néanmoins à expliquer ces deux valeurs pour expliquer certaines des optimisations qui ont pu être tentées sur la mémoire liée aux bitsets.

C – Optimisation du balayage : le parcours boustrophédon

Le balayage standard au sein d'une tuile suit l'ordre de lecture (ligne par ligne, de gauche à droite). Cette méthode est inefficace pour les configurations comme spirals où les grains peuvent se propager « à contre-courant ». Nous utilisons donc un parcours boustrophédon :

Listing 14 – Pseudo-code du balayage bidirectionnel (Boustrophédon)

```
1 Fonction do_tile_bidi (tuile)
2   Repeter 3 fois:
3     // Passe 1 : Aller (Haut-Gauche vers Bas-Droite)
4     Pour chaque ligne i de 0 a height-1:
5       Pour chaque colonne j de 0 a width-1:
6         Si cellule(i, j) >= 4: dispatcher grains;
7
8     // Sortie precocce si aucune modification detectee
9     Si aucun changement: Retourner STABLE;
10
11    // Passe 2 : Retour (Bas-Droite vers Haut-Gauche)
12    Pour chaque ligne i de height-1 a 0:
13      Pour chaque colonne j de width-1 a 0:
14        Si cellule(i, j) >= 4: dispatcher grains;
15
16    Si aucun changement: Retourner STABLE;
17  Fin Repeter
18 Fin Fonction
```

Cette approche permet à un grain situé en bas à droite d'une tuile de se propager vers le haut et la gauche en une seule itération de tuile, évitant d'attendre l'itération globale suivante pour remonter.

D – Fonctions do_tile et compute, ssandPile 1024x1024

Listing 15 – ssandPile : do_tile 1024 – ssandPile.c

```
1
2 int ssandPile_do_tile_opt_border (int x, int y, int width, int height)
3 {
4   unsigned int offset = DIM - width;
5   int diff = 0;
6
7   TYPE *restrict in_cell = table_cell (TABLE, in, y, x);
8   TYPE *restrict out_cell = table_cell (TABLE, out, y, x);
9
10  for (int i = 0; i < height; i++) {
11    int is_border_up = (i == 0);
12    int is_border_down = (i == (height - 1));
```

```

13
14 for (int j = 0; j < width; j++) {
15     int is_border_left = (j == 0);
16     int is_border_right = (j == (width - 1));
17
18     unsigned int b = (*(in_cell - 1) >> 2);
19     unsigned int old = (*(in_cell) & 3);
20     unsigned int a = (*(in_cell + 1) >> 2);
21     unsigned int c = (*(in_cell - DIM) >> 2);
22     unsigned int d = (*(in_cell + DIM) >> 2);
23
24     unsigned int new = a + b + c + d + old;
25     unsigned loc_diff = SANDPILE_BORDER_SET_SELF (new != old);
26
27     int up_flag = SANDPILE_BORDER_SET_UP (loc_diff, is_border_up);
28     int down_flag = SANDPILE_BORDER_SET_DOWN (loc_diff, is_border_down);
29     int left_flag = SANDPILE_BORDER_SET_LEFT (loc_diff, is_border_left);
30     int right_flag = SANDPILE_BORDER_SET_RIGHT (loc_diff, is_border_right);
31
32     diff |= loc_diff | up_flag | down_flag | left_flag | right_flag;
33
34     *out_cell = new;
35     in_cell++;
36     out_cell++;
37 }
38
39 in_cell += offset;
40 out_cell += offset;
41 }
42
43 return diff;
44 }

```

Ici, SANDPILE_BORDER_SET_* sont des macros permettant d'utiliser les flags de bordures comme définit en partie 2 1.

Listing 16 – ssandPile : compute 1024 – ssandPile.c

```

1 unsigned ssandPile_compute_omp_lazy_border (unsigned nb_iter)
2 {
3     unsigned it;
4     for (it = 1; it <= nb_iter; it++) {
5         int change = 0;
6
7         bitset *restrict sets = &tileset_at(TILESET, 0);
8
9         #pragma omp parallel shared(TABLE)
10        {
11            tileset_t curr = tilesets [omp_get_thread_num ()];
12            tileset_mark_empty (curr);
13            #pragma omp for schedule(static)
14            for (unsigned i = 0; i < TOTAL_NB_SETS; i++) {
15                unsigned char in_set = PER_SET [i];
16                unsigned tile_y = i / SETS_PER_ROW;
17                unsigned set_x = i - tile_y * SETS_PER_ROW;
18                bitset set = sets[i];
19
20                for (unsigned char k = 0; k < in_set; k++) {
21                    unsigned pos = __builtin_clzll (set);
22                    set -= bitset_at (pos);
23
24                    tile t;
25                    t.tx = BITSET_SIZE_MUL (set_x) + pos;
26                    t.ty = tile_y;
27

```

```

28     unsigned long x, y;
29     x = t.tx * TILE_W;
30     y = t.ty * TILE_H;
31
32     int y_0    = (y == 0);
33     int x_0    = (x == 0);
34     int y_end  = (y + TILE_H == DIM);
35     int x_end  = (x + TILE_W == DIM);
36
37     int loc_change = do_tile (x + x_0, y + y_0, TILE_W - x_end - x_0,
38                             TILE_H - y_end - y_0);
39
40     tile self = t;
41     tile up   = {.tx = t.tx, .ty = t.ty - 1 + y_0};
42     tile down = {.tx = t.tx, .ty = t.ty + 1 - y_end};
43     tile left  = {.tx = t.tx - 1 + x_0, .ty = t.ty};
44     tile right = {.tx = t.tx + 1 - x_end, .ty = t.ty};
45
46     int mark_self = SANDPILE_BORDER_GET_SELF (loc_change);
47     int mark_up   = SANDPILE_BORDER_GET_UP   (loc_change);
48     int mark_down = SANDPILE_BORDER_GET_DOWN (loc_change);
49     int mark_left  = SANDPILE_BORDER_GET_LEFT (loc_change);
50     int mark_right = SANDPILE_BORDER_GET_RIGHT (loc_change);
51
52     tileset_mark_at (curr, mark_self, self);
53     tileset_mark_at (curr, mark_up, up);
54     tileset_mark_at (curr, mark_down, down);
55     tileset_mark_at (curr, mark_left, left);
56     tileset_mark_at (curr, mark_right, right);
57 }
58
59 sets[i] = 0;
60 }
61 }
62
63 swap_tables ();
64
65 change = tileset_merge (TILESET, tilesets, num_threads);
66 if (change == 0)
67     break;
68 }
69
70 if (it <= nb_iter)
71     return it;
72 return 0;
73 }

```